Chapter 1: Foundations of Distributed Systems

A distributed system consists of autonomous computing nodes that communicate over a shared network to coordinate their actions. Unlike centralized systems, distributed systems face independent node failures, network partitions, and lack of a global clock. The fundamental model assumes message passing with arbitrary network delays and potential packet loss. System models can be synchronous, where bounds on processing and network delays are known, or asynchronous, where no timing assumptions hold. Fault tolerance requires distinguishing between crash-stop failures, crash-recovery failures, and arbitrary Byzantine failures. Replication is the primary technique for high availability, allowing a service to remain operational even when individual nodes crash.

Chapter 2: The CAP Theorem and PACELC

Formulated by Eric Brewer and formally proven by Seth Gilbert and Nancy Lynch, the CAP theorem states that a distributed data store can simultaneously provide at most two of three guarantees: Consistency, Availability, and Partition Tolerance. Since network partitions are inevitable in real-world physical networks, systems must choose between consistency (CP) or availability (AP) during a partition. Daniel Abadi extended this trade-off with the PACELC theorem: If there is a Partition (P), how does the system trade off Availability (A) and Consistency (C); Else (E), when the system is running normally without partitions, how does the system trade off Latency (L) and Consistency (C)? For example, Dynamo and Cassandra choose PA/EL, optimizing for availability and low latency, whereas Bigtable and Spanner prioritize PC/EC.

Chapter 3: Consensus and the Paxos Protocol

Consensus is the fundamental problem of agreeing on a single data value among multiple distributed nodes in an asynchronous network. Leslie Lamport introduced the Paxos family of protocols to solve consensus in the presence of crash failures. Basic Paxos operates in two distinct phases: Phase 1 (Prepare and Promise) and Phase 2 (Accept and Accepted). A proposer node sends a Prepare request with a unique ballot number to a majority quorum of acceptors. If a majority responds with a Promise not to accept proposals with lower ballot numbers, the proposer issues an Accept request. Multi-Paxos optimizes this protocol by electing a stable leader, amortizing the cost of Phase 1 across multiple log entries.

Chapter 4: The Raft Consensus Algorithm

Designed by Diego Ongaro and John Ousterhout, Raft was created as an understandable alternative to Multi-Paxos. Raft decomposes consensus into three independent subproblems: Leader Election, Log Replication, and Safety. At any time, a node is in one of three states: Leader, Follower, or Candidate. Time is divided into arbitrary terms identified by monotonic integers. If a follower detects leader election timeout, it becomes a candidate. The candidate increments its current term, votes for itself, and broadcasts RequestVote RPCs to all peers. Once elected, the leader accepts client commands, appends them to its log, and sends AppendEntries RPCs to replicate entries across a quorum.

Chapter 5: Logical Time and Vector Clocks

In distributed systems, physical clocks cannot be perfectly synchronized due to clock drift and variable network latency. Leslie Lamport introduced logical clocks to define a partial ordering of events based on the happened-before relation (denoted as ->). Lamport timestamps increment on local events and update to max(local, received) + 1 upon receiving messages. However, Lamport clocks cannot distinguish between causally related events and concurrent events. Vector clocks solve this limitation by maintaining a vector of integers, where each entry represents the local clock of a specific node. Two events are concurrent if neither vector dominates the other, enabling systems like Riak to detect conflicting updates.

Chapter 6: Byzantine Fault Tolerance

Byzantine fault tolerance (BFT) addresses systems where faulty nodes may exhibit arbitrary behavior, including malicious attacks and lying. The Byzantine Generals Problem illustrates the challenge of reaching agreement among commanders who may be traitors. In a system with f Byzantine nodes, consensus is impossible unless the total number of nodes n satisfies n >= 3f + 1. Miguel Castro and Barbara Liskov introduced Practical Byzantine Fault Tolerance (PBFT), which operates in an asynchronous network. PBFT uses three phases: Pre-Prepare, Prepare, and Commit, and relies on cryptographic signatures or message authentication codes (MACs). Modern blockchain consensus mechanisms, such as Tendermint and PBFT-based state machine replication, build directly upon these foundations.

Chapter 7: Distributed Transactions and Two-Phase Commit

When an atomic transaction spans multiple autonomous storage nodes, distributed commit protocols ensure atomicity across all participants. The Two-Phase Commit (2PC) protocol coordinates transactions using a designated coordinator and participating resource managers. In Phase 1 (Prepare phase), the coordinator sends a PREPARE message to all participants, asking whether they can commit the transaction. Each participant checks local constraints, writes uncommitted changes to a write-ahead log (WAL), and votes either VOTE_COMMIT or VOTE_ABORT. In Phase 2 (Commit phase), if all participants vote VOTE_COMMIT, the coordinator broadcasts a GLOBAL_COMMIT; otherwise, it sends GLOBAL_ABORT. A major limitation of 2PC is that it is a blocking protocol: if the coordinator crashes during Phase 2, participants remain blocked.

Chapter 8: Eventual Consistency and CRDTs

Eventual consistency is a weak consistency model where, in the absence of new updates, all replicas eventually converge to the same value. To resolve conflicting concurrent updates without coordination or consensus locks, Conflict-free Replicated Data Types (CRDTs) were introduced. CRDTs come in two varieties: State-based (CvRDTs) and Operation-based (CmRDTs). State-based CRDTs propagate full state and merge replicas using a mathematically proven join-semilattice with a least upper bound operator. Because merge operations are associative, commutative, and idempotent, convergence is mathematically guaranteed regardless of message reordering. Examples include G-Counters (grow-only counters), PN-Counters (positive-negative counters), and Observed-Removed Sets (OR-Sets).